

Riprendiamo da qui — nota per la prossima sessione

Aggiornato: 16/08/2026.

Dove siamo (tutto su `main`)

- **Indice di tutto il materiale:** `02 INDICE.md` in radice — va tenuto aggiornato a ogni aggiunta o cambio. (Documenti tecnici rinumerati `00/01/02` secondo `REGOLE-DOCUMENTI.md`.)
- **Manuale v0.20** — teoria, lettera a mano firmata "Nicola", schede, i 4 capitoli di costruzione degli esercizi 1-2-3-4 (Es4 = vite e game over), progetto boss. Nel Cap. 3 i mattoncini booleano/stato e visible. Ora ha un **Sommario cliccabile** dopo la copertina (+ segnalibri laterali nel PDF) e **tutte** le foto: ogni esercizio ha sia la foto dell'ambiente sia quella del gioco.
- **Super-guida** `CORSO-COMPLETO-v1.2.pdf` — tutti i documenti in un solo PDF, con indice cliccabile e segnalibri (si genera con `manuale/_build/volume_pdf.py`). C'è anche `00 CORSO.md` / `00 CORSO.pdf`, il quadro completo stampabile.
- **Eserciziario v0.15** (PDF `eserciziario-v0.15.pdf` generato) — esercizi 1-4 (4 livelli + foto del risultato) + BOSS, ognuno con la "Guida passo passo — costruiamolo insieme" e la spiegazione del codice riga per riga. **Ogni passo della guida e' stato verificato sulle schermate reali di Godot 4.7.1 in italiano** (vedi nota qui sotto).
- **Esercizi funzionanti** in `esercizi/` (01 bottone, 02 quadrato, 03 moneta). Convenzione nomi: variabile finisce in "Var", nodo in "Scena".
- **Giochi:** `battaglia-navale-3d/` (Affonda la Bonomi) e `gioco-del-quindici/` (15-puzzle con la foto di Serena incorporata, tessere di legno driftwood in rilievo nella vaschetta, sfondo beige; menu solo 3x3/4x4). Entrambi pubblicati sul web (vedi sotto). Aggiunti quattro giochi nuovi: `acchiappa-la-talpa/` (talpe da toccare a tempo, `/talpa/`), `schiva-gli-asteroidi/` (navetta che schiva i sassi, `/asteroidi/`), `rompi-i-mattoni/` (breakout, `/mattoni/`) e `torta-in-faccia/` (tiro alla torta sul bersaglio-faccia, `/torta/`).
Tutti e quattro ora hanno i suoni (presa, splat, rimbalzo/mattone, boom): file WAV in ogni cartella, agganciati via `AudioStreamPlayer` nel codice.

- **Classifica online condivisa** sui quattro giochi arcade (`classifica.gd` , identico in ogni cartella): a fine partita legge/scrive i primi 10 su un **Firestore Realtime Database** via REST. A fine partita appare un overlay con la classifica; se entri nei primi 10 scrivi il nome con una **tastiera a bottoni disegnata nel gioco** (la tastiera di Android nella LineEdit di Godot e `window.prompt` NON erano affidabili → risolto così). Ogni gioco usa una sua chiave (`gioco = "talpa"/"asteroidi"/"mattoni"/"torta"`).
- **Database:** progetto Firestore `esempio1` (id `nicolaregge2` , creato da Nicola nel suo account Google, piano gratuito Spark). URL nel codice: `https://nicolaregge2-default-rtdb.europe-west1.firebaseio.com` . Regole pubbliche permanenti (`.read / .write: true`) → chiunque gioca può scrivere. Claude da qui NON raggiunge il database (proxy lo blocca): il collaudo va fatto sul telefono.
- **Anti-cache:** `/tmp/bust.py` inietta in ogni `docs/*/index.html` una patch che ricarica `index.pck` sempre fresco (`Date.now()` a ogni apertura), così i **link puliti** (senza `?v=`) mostrano da soli l'ultima versione. Da rilanciare dopo ogni export che sovrascrive l'`index.html`.
- **Release v1.0** pubblicata (versione congelata per i ragazzi). La v1.1 è da pubblicare quando Nicola vuole (il tag lo crea lui dal browser, Claude non può).
- Tutte le foto degli esercizi sono in `manuale/immagini/` (es1-ambiente, es1-gioca, es2-ambiente, es2-gioca, es3-ambiente, es3-gioca, es4-ambiente, es4-gioca).

Verifica delle guide sull'interfaccia REALE di Godot (metodo, da ricordare)

Le coordinate e i nomi dei pulsanti nelle guide passo-passo **non si scrivono a memoria**: si verificano guardando l'interfaccia vera. In questo ambiente Claude **puo' far girare Godot 4.7.1 con interfaccia grafica** e fotografarla:

- schermo virtuale **Xvfb** (`:99`), Godot con `--display-driver x11 --rendering-driver opengl3` , screenshot con **Pillow ImageGrab**;
- lingua editor in italiano: in `~/ .config/godot/editor_settings-4.7.tres` , `interface/editor/localization/editor_language = "it"` ;
- i **clic sui pulsanti** (Button) funzionano via XTEST; i clic sugli elementi ad **albero** (Scene tree, file browser) e la **tastiera** NON registrano bene senza window manager → per quei passi si aprono progetti già pronti o si usano schermate note. Le finestre/menu a comparsa vanno bene se sono dialoghi.
- Regola: se non riesco a vedere un passo con i miei occhi, **non lo affermo**: lo scrivo in modo prudente o chiedo a Nicola una schermata. Mai indovinare.

Cose confermate sulle schermate reali (Godot 4.7.1 IT): `Crea / Importa` in alto a SINISTRA nel Gestore progetti; finestra nuovo progetto con "Nome del progetto" in cima, "Percorso del progetto" sotto, Renderer con "Compatibilit ", pulsante "Crea" + casella "Modifica ora" (non "Crea e modifica"); pannello Scena vuoto con i pulsanti radice "Scena 2D / Scena 3D / Interfaccia utente / Altro nodo"; finestra "Crea un nuovo Node" con Cerca/Corrispondenze/Preferiti/Recenti; lo script si attacca con l'icona "Allega uno script" (foglio con + verde) nella barra del pannello Scena; l'import apre una finestra file (scegli `project.godot` → "Apri").

Pubblicazione web dei giochi (GitHub Pages) — COME FUNZIONA

Regola operativa importante (era fonte di confusione, ora   chiara e va ricordata):

- **Sorgente di Pages = "GitHub Actions"** (Settings → Pages → Source). NON "Deploy from a branch". (Prima puntava al ramo vecchio `gioco-battaglia-3d`, perci  i giochi nuovi davano 404: risolto.)
- Un workflow, `.github/workflows/deploy-pages.yml`, pubblica la cartella `docs/` del ramo `main` a ogni push su main (e su avvio manuale).
- **Claude pu  pubblicare/aggiornare da solo** (avvia e segue il workflow da qui): Nicola non deve toccare impostazioni.
- I giochi web stanno in `docs/<gioco>/`. Link attuali:
 - * Gioco del quindici: `https://nicolaregge-pulse.github.io/corso-godot/quindici/`
 - * Affonda la Bonomi: `https://nicolaregge-pulse.github.io/corso-godot/`
- Per mettere un gioco sul web: esportarlo da Godot in `docs/<gioco>/` (preset Web, **senza thread**, cos  va anche su iPhone), poi push su main → si pubblica da solo. In questa sessione l'export l'ha fatto Claude scaricando Godot 4.7.1 + i modelli di esportazione nell'ambiente (i modelli non restano tra le sessioni).

Cassaforte "Frau Doktor Bonomi" (app personale di Nicola) — STATO

App web che cifra un file con password (AES-256, tutto sul dispositivo, niente in rete). Stile steampunk/ottone. Sorgente di verit : `docs/cassaforte/index.html` (la lavorazione sta nello scratchpad di sessione, che   effimero). Online: `https://nicolaregge-pulse.github.io/corso-godot/cassaforte/`.

Il vincolo iPhone (confermato, da NON dimenticare): quando iPhone apre un file "al volo" (anteprima Quick Look di WhatsApp/Mail) **NON esegue il JavaScript**. Perciò il file `[nome].html` che "si apre da solo" funziona su **Android/PC/Mac** ma **NON** in anteprima su iPhone (compare la cassaforte ma il tasto non fa niente: manca proprio l'esecuzione del codice). Il codice gira solo in una **vera pagina web aperta in Safari**.

Strada che su iPhone funziona senza limiti di dimensione (foto comprese):

1. Salvare il file: in WhatsApp/Mail toccare il file → icona **Condividi** in alto a destra → **Salva su File** → **Salva**.

2. Aprire in Safari la pagina Cassaforte → linguetta **Apri** → **Scegli file** → prendere il `[nome].html` salvato → password → **Apri e scarica**.

Il "link con i dati dentro" (un tocco) è stato scartato: ha un limite di dimensione (~40 KB), Nicola vuole **senza limiti**.

Migliorie già pubblicate (PR #56): l'anteprima del file su iPhone ora mostra le istruzioni chiare (Salva su File + link) invece della cassaforte "morta"; aggiunta la rotella con la percentuale mentre prepara il file.

REGOLA (decisa da Nicola): le cose "coi ragazzi" si fanno IN CLASSE

Tutto ciò che riguarda gli studenti — creare gli **account GitHub**, il **primo giro di consegna**, l'**invito alla classe** — si fa **con i ragazzi davanti, a scuola (da settembre), NON prima**. Non prepararle né sollecitarle in anticipo: il materiale è già pronto (`consegne/`), basta usarlo quando la classe c'è.

Cosa manca (prossimi passi)

Con la classe (a settembre, vedi regola sopra): account ragazzi · primo giro di consegna · invito alla classe.

Opzionali quando Nicola vuole (non urgenti, si possono fare anche prima):

- **Release** stabile aggiornata per i ragazzi (tag dal browser) quando si vuole congelare il materiale nuovo (giochi, kit consegne, manuale 0.19).
- Foto `es4-gioca.png` da inserire nel capitolo Esercizio 4 del manuale.
- Ritocchi ai giochi: suono (splat torta, presa talpa) e schermata iniziale.
- Prova Cassaforte su iPhone con Serena (sospesa: da remoto e' scomoda).

Documenti consegnabili anche in PDF (regola di Nicola)

Le **guide/documenti consegnabili** vanno prodotti **anche in PDF**, non solo in MD. Strumento: `manuale/_build/guida_pdf.py` (MD → HTML pulito) + `guida_pdf.js` (HTML → PDF con Chromium). Uso:

```
python3 manuale/_build/guida_pdf.py <guida.md> poi
node manuale/_build/guida_pdf.js <guida.guida.html> <guida.pdf> .
```

Gia' fatti: `consegne/CREA-ACCOUNT-GITHUB.pdf`, `consegne/INVITO-ALLA-CLASSE.pdf`. Il manuale e l'eserciziario hanno il loro build dedicato (`genera_pdf.py`).

Kit di consegna dei ragazzi — FATTO (in `consegne/`)

Ogni consegna = una cartella con: `scheda.md` (modulo: cognome, nome, classe, esercizio/titolo, data, "cosa ho fatto con parole mie", "il mio tocco", "dove mi sono bloccato"), il **codice**, le **immagini**.

- Struttura per gestire più classi e più anni:

```
consegne/<anno-classe>/<cognome-nome>/<esercizio>/...
```

- `consegne/_MODELLO/` = kit vuoto che i ragazzi copiano (scheda, ISTRUZIONI con anche "come viene valutata la tua consegna", cartella immagini).

- `consegne/_PROF/` = solo prof: `COME-CORREGGO.md` (giro completo, i 4 segnali, la scala del voto APPROVATA) e `valutazione-MODELLO.md` .

- **Esempio già compilato:** `consegne/2026-2027-1informatica/rossi-mario/es1-bottone/` (scheda + main.gd + screenshot + valutazione) come modello per i ragazzi.

- I **4 segnali** valutati: consegna completa · funziona · il tuo tocco · lo sai spiegare con parole tue. Scala voto CONFERMATA: 9-10 tutto ok e lo sa spiegare; 7-8 funziona e tocco, spiegazione incerta; 6 tocco minimo o spiegazione debole; 5 incompleta o non funziona ma tentativo vero → si può RI-CONSEGNARE. Si parte dall'alto, giudizio sempre incoraggiante, ultima parola al prof.

- Giro: ragazzo consegna → Claude corregge → **voto = numero + breve giudizio** → Claude aggiunge una pagina al **manuale personale** dello studente.

- I ragazzi lavorano con **Gemini**; Claude corregge e compila i loro manuali.

Idee più avanti

- Impaginare il **quaderno-studente** in PDF (base del manuale personale).
- Riordinare gli esercizi per difficoltà; aggiungerne altri.
- Gioco del quindici: eventuale vera texture di legno; Esercizio 4.

Preferenze da ricordare **SEMPRE** (decise da Nicola)

- Aspetto **sobrio, serio, professionale**: niente icone/emoji. Unica eccezione: i **4 livelli d'aiuto** restano colorati (verde/giallo/arancione/rosso).

- **Niente parentesi** nei **titoli** e nelle **scritte in grassetto** (nel testo discorsivo e nel codice possono restare).
- Il libro **parla ai ragazzi** ("tu"), non è una riflessione tra docente e Al.
- **Changelog** solo nel sorgente `.md`, **non** nel PDF. Versione nel footer.
- Ogni PDF ha la **versione nel nome file**; mai riusare una versione.
- **Mai far salvare/consegnare un nome di file che esiste già**: ogni file consegnato ha nome unico con versione; non re-inviare mai un file identico che Nicola ha già (causa "file già esistente" quando salva). Regola sacra.
- Con Nicola e i ragazzi: **tutto visuale** (GitHub Desktop / browser), mai la riga di comando.
- **COORDINATE COMPLETE, SEMPRE (regola vincolante)**. Prima di ogni azione dico, in quest'ordine: 1) **quale APPLICAZIONE** (es. `[APP – GitHub Desktop]`, `[APP – Godot]`, `[BROWSER]`); 2) **quale FINESTRA / SCHEDA**; 3) **quale MENU** → **voce** → **sotto-voce**; 4) **in quale AREA** (pannello a sinistra, barra in alto, ecc.); 5) **l'AZIONE esatta**, una alla volta. Mai un comando "nudo" senza dire DOVE va fatto. Testi/valori da copiare sempre in blocco di codice.
- **Descrizione del commit sempre pronta da me**: ogni volta che so che Nicola dovrà fare un commit (file generati da Godot, modifiche nostre, PDF nuovi), gli fornisco IO il testo del campo Summary in un blocco da copiare, senza che lo debba chiedere. Anticipare, non aspettare.
- **PRECISIONE MICIDIALE nelle guide passo-passo (regola vincolante)**. Alcuni ragazzi "non vanno oltre un centimetro": non colmano i vuoti, fanno ESATTAMENTE ciò che c'è scritto. Ci sono anche ragazzi con disabilità: un termine ambiguo o un passo saltato è un muro, non un fastidio. Quindi ogni guida operativa deve:
 - usare i **nomi esatti come compaiono a schermo** (finestre e pulsanti: es. finestra "Crea un nuovo Node", pulsante "Scena 2D");
 - **una micro-azione per riga**, numerata; mai due cose nella stessa frase;
 - **mai** "scegli/vai/sistema" senza dire DOVE e COSA cliccare;
 - **sempre lo stesso termine** per la stessa cosa (niente sinonimi);
 - **guidare lo sguardo** (dire dove guardare prima di cosa fare);
 - meglio ridondanti che ambigui. Scrivere immaginando cosa vede LO STUDENTE, non cosa so io.